

```
"""Kylie Alberts
I affirm that I have carried out the attached academic endeavors with full
academic honesty,
in accordance with the Union College Honor Code and the course syllabus."""
```

```
import random
BOARDSIZE=4
```

```
# creates a board filled with question marks (?). The board will be
# a 4X4 grid by default, but you should write this so the size is easy to
# change.
```

```
def createBoard():
    board=[]
    for row in range(0,BOARDSIZE):
        board.append([])
        for col in range(0,BOARDSIZE):
            board[row].append("?")
    return board
```

```
# Shake up the dice to make a new Boggle board. That is, given a board
# (2D list) and a string of capitalized letters, use the letters in
# the string to initialize each space on the board. For example,
# if letters = "ETCOLCVEEMPASHLI", then the board should be set to
# [["E","T","C","O"],["L","C","V","E"],
#  ["E","M","P","A"],["S","H","L","I"]]
# The board should be returned.
```

```
def shakeUp(board, letters):
    count=0
    for row in range(0,len(board)):
        for col in range(0,len(board)):
            board[row][col]=letters[count]
            count=count+1
    return board
```

```
# returns either a random string of letters or a set string of letters that
# will be used to initialize the board
```

```
#DEBUG can be set to true or false depending on if you want to debug or not
```

```
def makeBoardString():
    DEBUG=False
    string=""
```

```

if DEBUG is True:
    letters="TIPEYECVLSRAWOTU"
    return letters
else:
    for index in range(0,BOARDSIZE**2):
        letter=chr(random.randint(65,90))
        string=string+letter
    return string

def pretty_print(board):
    """Given a two dimensional list L, print it
    like a grid (with rows and columns)"""
    for row in range(0,len(board)):
        row_string=make_a_row(board[row])
        print(row_string+"\n")

def make_a_row(one_row):
    """Given a single row (1-D list)
    return a string version of it.
    Example: if one_row is [1,2,3],
    this returns "1 2 3"
    """
    string=""
    for index in range(0,len(one_row)):
        string=string+str(one_row[index])+" "
    return string

def letter_to_number(row,col):
    #converts row and column of a letter to a corresponding number on the
    board
    start_of_row=row*BOARDSIZE
    number=start_of_row+col
    return number

# Given a board (2D list), returns a dictionary that maps letters to lists
# of board numbers where they appear. Every capital letter is represented
# in the dictionary. If a letter does not appear on the board, its board
# number list will be empty.
# Ex: {"K": [8, 12]} means K can be found at board numbers 8 and 12

```

```

def createDict(board):
    d={}
    for index in range(65,91):
        d[chr(index)]=[]
    for row in range(0,len(board)):
        L=[]
        for col in range(0,len(board[row])):
            number=letter_to_number(row,col)
            letter=board[row][col]
            d[letter].append(number)
    return d

#takes a letter and finds a list of board numbers
def find_boardnum(letter, letterdict):
    d=letterdict
    L=[]
    if letter in d:
        L=d[letter]
    return L

# Given a list of board numbers for the first letter of a word,
# return a list that contains each board number in its own list
# So [3,4,5] will cause [ [3], [4], [5] ] to be returned.
# PITFALL ALERT: this function should create a BRAND NEW LIST to return.
# first_letter_list should not be altered by this function.
def initializeChains(first_letter_list):
    newlist=[]
    for index in range(0,len(first_letter_list)):
        number=first_letter_list[index]
        newlist.append([number])
    return newlist

def converts_number_to_coord(number):
    #converts a given number to its coordinate on the board
    row=int(converts_to_row(number))
    col=int(converts_to_col(number))
    coord=[row,col]
    return coord

```

```

def converts_to_row(number):
    #converts a given number to its row on the board
    row=number/BOARDSIZE
    return row

def converts_to_col(number):
    #converts a given number to its column on the board
    col=number%BOARDSIZE
    return col

# given 2 board numbers on the Boggle board (each between 0-15 for 4X4),
# return True if the board numbers are adjacent and False otherwise.
# There is no wrap-around. They are adjacent if the distance
# between rows and the distance between cols are both no more than 1.
# Remember, identical board numbers are not adjacent -- test for this!
def isAdjacent(boardNum1, boardNum2):
    coord1=converts_number_to_coord(boardNum1)
    coord2=converts_number_to_coord(boardNum2)
    if abs(coord1[1]-coord2[1])<=1 and coord1 != coord2 and
abs(coord1[0]-coord2[0])<=1:
        return True
    else:
        return False

# given a list of board numbers of the next letter and the current
# set of chains, see if any board number in the list is adjacent to
# the last board number in any of the chains. If so, append & put it
# into a new list (called nextChains in the Powerpoint slides).
# The new board number cannot be a repeat of a board number already
# in the chain! nextChains is returned.
def connect(boardNumberList, chains):
    nextChains=[]
    for row in range(0,len(boardNumberList)):
        for col in range(0,len(chains)):
            boardnum1=boardNumberList[row]
            newchainboardnum=chains[col]
            if newchainboardnum != [] and
isAdjacent(boardnum1[0],newchainboardnum[-1]) and not boardnum1[0] in
newchainboardnum:
                nextChainspart=chains[col]+boardnum1
            else:

```

```

        nextChainspart=[]
        nextChains.append(nextChainspart)
    return nextChains

# returns True if word on board, else False. letterDict is
# a dictionary that maps letters to lists of board numbers showing
# where that letter occurs on board
def find(word, letterDict):
    word=word.upper()
    firstletter=word[0]
    list_of_boardnum=find_boardnum(firstletter,letterDict)
    chains=initializeChains(list_of_boardnum)
    count=1
    while chains and count<len(word):
        L=[]
        boardnum=find_boardnum(word[count], letterDict)
        L.append(boardnum)
        L=initializeChains(L[0])
        chains=connect(L, chains)
        count=count+1
    for index in range(0,len(chains)):
        if chains[index] != []:
            return True
    return False

# given the name of a word file (a String like "smallWordFile.txt")
# and the letter-to-boardNums dictionary, go through each
# word in the word file, and if on the board and >= 4 letters,
# append the word to a list. That list is returned.
def computerPlays(wordFilename, letterDict):
    infile=open(wordFilename,"r")
    filelist=file2list(wordFilename)
    L=[]
    for word in filelist:
        if len(word) >= 4 and find(word, letterDict):
            L.append(word)
    infile.close()
    return L

```

```

# builds a list out of a file and returns it. This way, both players
# will have their words in a list. The parameter is the name of a
# file (a string)
def file2list(filename):
    L=[]
    infile=open(filename, "r")
    line=infile.readline()
    for line in infile:
        line=line.strip()
        L.append(line)
    infile.close()
    return L

# return a list of all elements in list1 that are not in list2.
def findUnique(list1,list2):
    uniquelist=[]
    for index in list1:
        if index not in list2:
            uniquelist.append(index)
    return uniquelist

# print results of this round including the list of unique words the
# human found (uniqueHumanList), the list of unique words the
# computer found (uniqueCompList), and the updated scores. Please
# list each word on a separate line; don't just print the list.
# This function does not update the scores. That's done in the play
# function. This is just pretty-printing all of the stats for the round.
def printResults(uniqueHumanList, uniqueCompList, humanScore, compScore):
    print("Your Unique List:")
    for row in range(0, len(uniqueHumanList)):
        rowstringhuman=uniqueHumanList[row]
        print(str(rowstringhuman) + "\n")
    print("Computer's Unique List:")
    for row in range(0, len(uniqueCompList)):
        rowstringcomp=uniqueCompList[row]
        print(str(rowstringcomp) + "\n")
    print("Your Score: " +str(humanScore))
    print("Computer's Score: " +str(compScore))

# Prints instructions for boggle and includes error checking
def showInstructionsIfNeeded():
    """This function asks the user if she would like to view

```

```

instructions on how to play the game. The user can type in 'y' or 'n' and
instructions will
be printed if the user enters 'y'. The function just ends if the user
enters 'n'. Nothing is
returned."""
    instructions= input("Would you like to view instructions on how to play
the game?")
    while instructions != "y" and instructions != "n":
        print("Invalid response. Please print either 'y' for yes or 'n' for
no.")
        instructions= input("Would you like to view instructions on how to
play the game?")
    if instructions == "y":
        print("A board will appear on the screen. You need to figure out
how many\
words you can make out of the words on the board. The computer will do the
same.\
You will get a point for every unique word you can identify that the
computer \
does not identify. Each word must have at least four letters. You must
reach above the set point value\
in order to win. If there is a tie, you will paly again until someone
wins.")

#Sets the difficulty level by specifying which of two word files the
# computer will use to find words. The computer will find more words
# using the larger word file. This function asks the user if they want the
# novice computer opponent or the expert one. Returns the name of the file
# ("smallWordFile.txt" or "bigWordFile.txt") to use. Use error-checking
# for the user's input. Do NOT open the file here. You'll do that in
# computerPlays.
def setDifficulty():
    answer=input("Would you like to set the difficulty to (E)asy or
(H)ard?")
    while answer != "E" and answer != "H":
        print("Invalid response. Please print either 'E' for easy or 'H'
for hard.")
        answer=input("Would you like to set the difficulty to (E)asy or
(H)ard?")
    if answer=="E":
        return "smallWordFile.txt"
    else:
        return "bigWordFile.txt"

```

```

#prints Welcome message
def welcome_message():
    print("Hi! You are about to play the game, Boggle! Enjoy!")

def points_until_done():
    answer=input("How many points would you like to play until?")
    return answer

# plays the game of Boggle
def play():
    welcome_message()
    showInstructionsIfNeeded()
    filename=setDifficulty()
    points=int(points_until_done())
    compScore=0
    humanScore=0
    while compScore == humanScore or (compScore < points and humanScore <
points):
        letters=makeBoardString()
        board=createBoard()
        newboard=shakeUp(board, letters)
        pretty_print(newboard)
        letterDict=createDict(board)
        input("Please put the words you found into 'player.txt'. Press
'enter' when complete.")
        computerList=computerPlays(filename,letterDict)
        humanList=file2list("player.txt")
        uniquehumanlist=findUnique(humanList,computerList)
        uniquecomplist=findUnique(computerList,humanList)
        humanroundscore=len(uniquehumanlist)
        comproundscore=len(uniquecomplist)
        compScore=compScore+comproundscore
        humanScore=humanScore+humanroundscore
        printResults(uniquehumanlist,uniquecomplist,humanScore,compScore)
    if compScore > humanScore:
        print("The computer won!")
    else:
        print("You won!")

def tester():

```



```

"""testing createBoard"""
print(createBoard())

#testing shakeUp
letters="LCEEFAECTLAMNOVS"

board=[["?","?","?","?"],["?","?","?","?"],["?","?","?","?"],["?","?","?","?"]]
newboard=shakeUp(board,letters)
print(newboard)

#""testing makeBoardstring""
print(makeBoardString())

#""testing pretty print board""
pretty_print(newboard)

#""make a row""
test=[1,2,3]
print(make_a_row(test))

#""letter to number""
print(letter_to_number(0,1))

#""createDict""
print(createDict(newboard))
list_dict=createDict(newboard)

#""find boardnum""
print(find_boardnum("A", list_dict))
firstlist=find_boardnum("A", list_dict)

#""italizechains""
print(initializeChains([3,4,5]))

#""is Adjacent""
print(isAdjacent(4,9))
print(isAdjacent(4,7))

#""connect""
print(connect([[13]], [[8],[9]]))

#""find""

```

```

print(find("loaf", list_dict))
print(find("nova", list_dict))
print(find("fleece", list_dict))
print(find("lava", list_dict))
print(find("alone", list_dict))

#""computerplays""
print(computerPlays("smallWordFile.txt", list_dict))

#""file2list""
print(file2list("smallWordFile.txt"))

#""findunique""
list1=[1,2,3]
list2=[2,3,4]
print(findUnique(list1,list2))

#""findResults""
uniquehumanlist=["hi","tie","my"]
uniquecomplist=["well","bear","bye"]
humanScore=4
compScore=6
printResults(uniquehumanlist,uniquecomplist,humanScore,compScore)

#""showinstructionsif needed""
showInstructionsIfNeeded()

#""setDifficulty""
setDifficulty()

def main():
    play()

main()
#tester()

```